1131軟體設計期末專案

minesweeper

P76131602吳睿秉

專案簡介

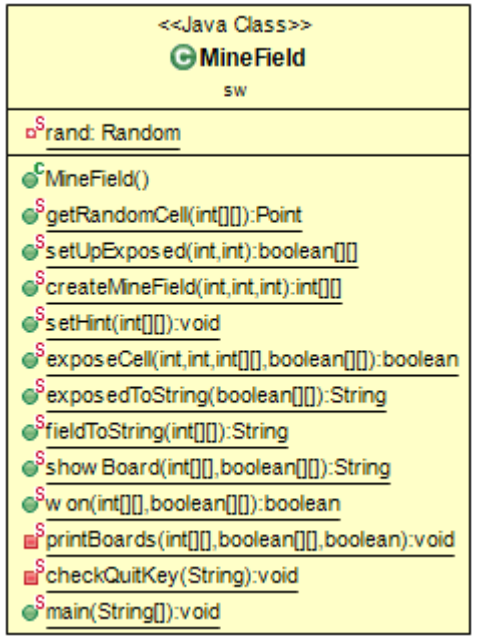
- 名稱:minesweeper
- 作者:Ben Wright
- 專案來源:https://github.com/ben31w/minesweeper
- 主題:踩地雷

- 重構結果: https://github.com/XTX7900XTX/minesweeper/tree/master

Function關係

鄰接矩陣		1	2	3	4	5	6	7	8	9	10	11	12
		main	setUpExposed	won	checkQuitKey	exposeCell	printBoards	fieldToString	showBoard	createMineField	getRandomCell	setHint	exposedToString
1	main	\	1	1	1	1	1	1	0	1	0	0	0
2	setUpExposed	0	\	0	0	0	0	0	0	0	0	0	0
3	won	0	0	\	0	0	0	0	0	0	0	0	0
4	checkQuitKey	0	0	0	\	0	0	0	0	0	0	0	0
5	exposeCell	0	0	0	0	\	0	0	0	0	0	0	0
6	printBoards	0	0	0	0	0	\	1	1	0	0	0	0
7	fieldToString	0	0	0	0	0	0	\	0	0	0	0	0
8	showBoard	0	0	0	0	0	0	0	\	0	0	0	0
9	createMineField	0	0	0	0	0	0	0	0	\	1	1	0
10	getRandomCell	0	0	0	0	0	0	0	0	0	\	0	0
11	setHint	0	0	0	0	0	0	0	0	0	0	\	0
12	exposedToString	0	0	0	0	0	0	0	0	0	0	0	\

Input
Output
Create Mine Field
Game Win Logic



MineField.java

exposedToString

main:

|- setUpExposed

|- won

|- checkQuitKey

|- exposeCell

|- createMineField

  |- getRandomCell

  |- setHint

|- printBoards

  |- showBoard

  |- fieldToString

|- fieldToString

MineField.java

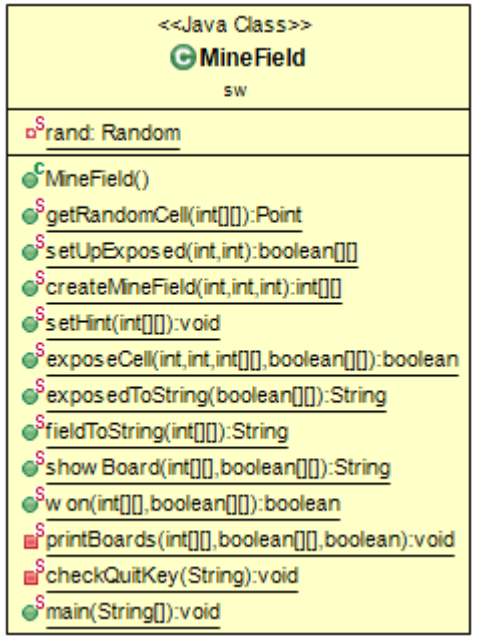
exposedToString

```java
    }


     * Returns a single string with the contents of the exposed array
    public static String exposedToString(boolean[][] exposed) {
        StringBuilder sb = new StringBuilder();
        for (int r = 0; r < exposed.length; r++) {
            for (int c = 0; c < exposed[r].length; c++) {
                if (exposed[r][c]) {
                    sb.append("T ");
                }
                else {
                    sb.append("F ");
                }
            }
            sb.append( System.lineSeparator() );
        }

        return sb.toString();
    }

```

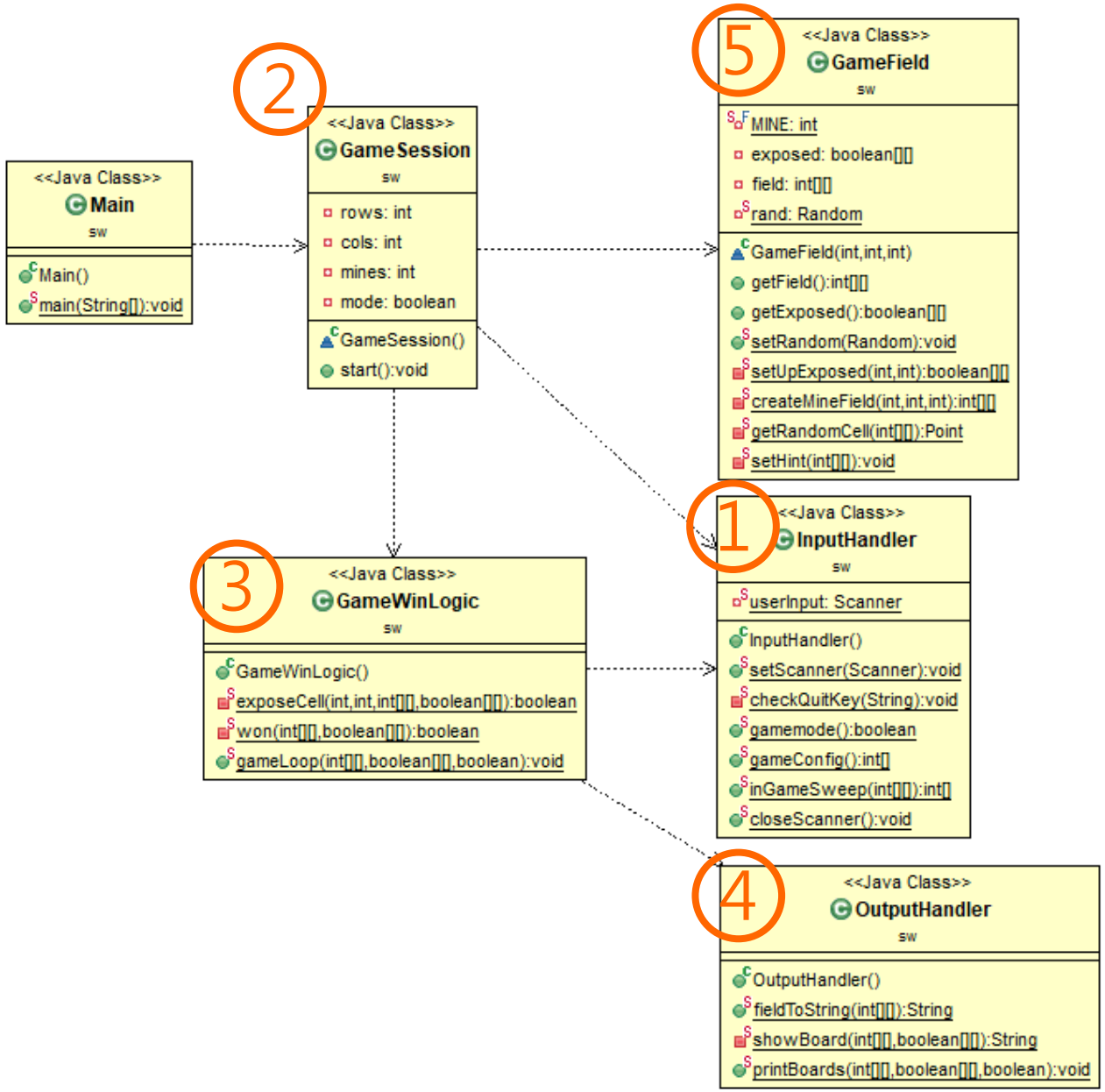
Refactoring

- **Before**

  Large Class



- **After**



Refactoring

Main:

- 建立遊戲
- 開始遊戲

GameSession:

- 建立地雷區
- 開始遊戲迴圈

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("Welcome to MineField. Enter 'q' at any time to quit.");
        GameSession newgame = new GameSession();
        newgame.start();
        System.out.println("Thank you for playing.");
    }
}
```

```java
public class GameSession {
    private int rows;
    private int cols;
    private int mines;
    private boolean mode;

    GameSession() {
        this.mode = InputHandler.gamemode();
        int[] gridSize = InputHandler.gameConfig();
        this.rows = gridSize[0];
        this.cols = gridSize[1];
        this.mines = gridSize[2];
    }

    public void start() {
        GameField field = new GameField(rows,cols,mines);
        GameWinLogic.gameLoop(field.getField(), field.getExposed(), mode);
        InputHandler.closeScanner();
    }
}
```

單元測試案例設計

- GameField.java
  GameField(int rows,int cols,int mines)
  - 1x2, number of mine = 1
  - 3x3, number of mine = 10
  - 10x10, number of mine = 50

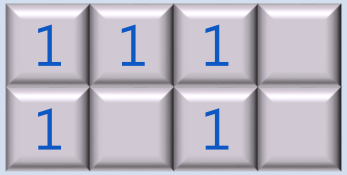
```java
final int ROWS = 1;
final int COLS = 2;
final int MINES = 1;
GameField.setRandom(new Random(0));
GameField testField = new GameField(ROWS,COLS,MINES);
int[][] expectField = {
        {1, -1}
    };
boolean[][] expectExposed = {
        {false, false}
};
```

```java
final int ROWS = 3;
final int COLS = 3;
final int MINES = 10;
GameField.setRandom(new Random(0));
GameField testField = new GameField(ROWS,COLS,MINES);
int[][] expectField = {
        {-1, -1, -1},
        {-1, -1, -1},
        {-1, -1, -1}
    };
boolean[][] expectExposed = {
        {false, false, false},
        {false, false, false},
        {false, false, false},
};
```

```java
final int ROWS = 10;
final int COLS = 10;
final int MINES = 50;
GameField.setRandom(new Random(0));
GameField testField = new GameField(ROWS,COLS,MINES);
int[][] expectField = {
        { 3,  -1,   2 ,  1,   2,  -1,   2,   3,  -1,   2},
        {-1,  -1,   4 ,  3,  -1,   2,   2,  -1,  -1,   2},
        {-1,   6,  -1 , -1,   2,   1,   2,   4,   4,   2},
        {-1,   5,  -1 ,  4,   2,   2,   4,  -1,  -1,   1},
        { 3,   6,  -1 ,  5,   4,  -1,  -1,  -1,   4,   2},
        {-1,  -1,  -1 , -1,  -1,  -1,   6,   4,  -1,   1},
        {-1,  -1,   8 , -1,  -1,  -1,  -1,   5,   4,   3},
        { 4,  -1,  -1 , -1,   7,   6,  -1,  -1,  -1,  -1},
        { 2,  -1,  -1 , -1,  -1,  -1,   6,  -1,  -1,   3},
        { 1,   2,   4 , -1,  -1,  -1,   4,  -1,   3,   1}
    };
boolean[][] expectExposed = {
        {false, false, false, false, false, false, false, false, false, false},
        {false, false, false, false, false, false, false, false, false, false},
        {false, false, false, false, false, false, false, false, false, false},
        {false, false, false, false, false, false, false, false, false, false},
        {false, false, false, false, false, false, false, false, false, false},
        {false, false, false, false, false, false, false, false, false, false},
        {false, false, false, false, false, false, false, false, false, false},
        {false, false, false, false, false, false, false, false, false, false},
        {false, false, false, false, false, false, false, false, false, false},
        {false, false, false, false, false, false, false, false, false, false}
};
```

單元測試案例設計

GameWinLogic.java
won()

描述	CASE	陣列內容		結果
不是地雷的都已開	空	field = {{}} expose = {{}}		W
	一格，打開不是地雷	field = {{0}} expose = {{T}}		W
	一格，是地雷沒打開	field = {{-1}} expose = {{F}}		W
	不是雷的全開	field = { { -1, -1, -1 }, { -1, -1, 3}} expose = {{ F, F, F}, {F, F, T}}		W
打開的任一是地雷 或 還有沒打開的格子	一格，雷被打開	field = {{-1}} expose = {{T}}		L
	一格，還沒打開	field = {{0}} expose = {{F}}		L
	有任一非雷還沒打開	field ={{ 1, 1, 1, 0 }, { 1, -1, 1, 0}} expose = {{ T, T, T, F}, { T, F, T, F }}		L

Smells

Magic Number -1 == 💣

```
for (int r = 0; r < field.length; r++) {
    for (int c = 0; c < field[r].length; c++) {
        field[r][c] = -1;
    }
}
```

```
private static final int MINE = -1;
```

Smells

```java
private static boolean won(int[][] field, boolean[][] exposed) {
    for (int r = 0; r < field.length; r++) {
        for (int c = 0; c < field[r].length; c++) {
            if (field[r][c] != -1 && !exposed[r][c]) {
                return false;
            }
            if (field[r][c] == -1 && exposed[r][c]) {
                return false;
            }
        }
    }
    return true;
}
```

```java
private static boolean won(int[][] field, boolean[][] exposed) {
    for (int r = 0; r < field.length; r++) {
        for (int c = 0; c < field[r].length; c++) {
            if ((field[r][c] != -1 && !exposed[r][c]) || (field[r][c] == -1 && exposed[r][c])) {
                return false;
            }
        }
    }
    return true;
}
```

```java
private static int[][] createMineField(int rows, int cols, int mines) {
    int[][] field = new int[rows][cols];

    if (mines >= rows * cols) {
        for (int r = 0; r < field.length; r++) {
            for (int c = 0; c < field[r].length; c++) {
                field[r][c] = MINE;
            }
        }
    }

    else if (mines > 0.5 * rows * cols) {
        for (int r = 0; r < field.length; r++) {
            for (int c = 0; c < field[r].length; c++) {
                field[r][c] = MINE;
            }
        }

        int minesRemoved = 0;
        while (minesRemoved < rows * cols - mines) {
            Point random = getRandomCell(field);
            while (field[random.x][random.y] != MINE) {
                random = getRandomCell(field);
            }

            field[random.x][random.y] = 0;
            minesRemoved++;
        }
    }
    else {
        int minesPlaced = 0;
        while (minesPlaced < mines) {
            Point random = getRandomCell(field);
            while (field[random.x][random.y] == MINE) {
                random = getRandomCell(field);
            }

            field[random.x][random.y] = MINE;
            minesPlaced++;
        }
    }

    setHint(field);

    return field;
}
```

Smells

```java
private static int[][] createMineField(int rows, int cols, int mines) {
    int[][] field = new int[rows][cols];

    if (mines >= rows * cols) {
        for (int r = 0; r < field.length; r++) {
            for (int c = 0; c < field[r].length; c++) {
                field[r][c] = MINE;
            }
        }
    }
    else {
        int minesPlaced = 0;
        while (minesPlaced < mines) {
            Point random = getRandomCell(field);
            while (field[random.x][random.y] == MINE) {
                random = getRandomCell(field);
            }

            field[random.x][random.y] = MINE;
            minesPlaced++;
        }
    }

    setHint(field);

    return field;
}
```

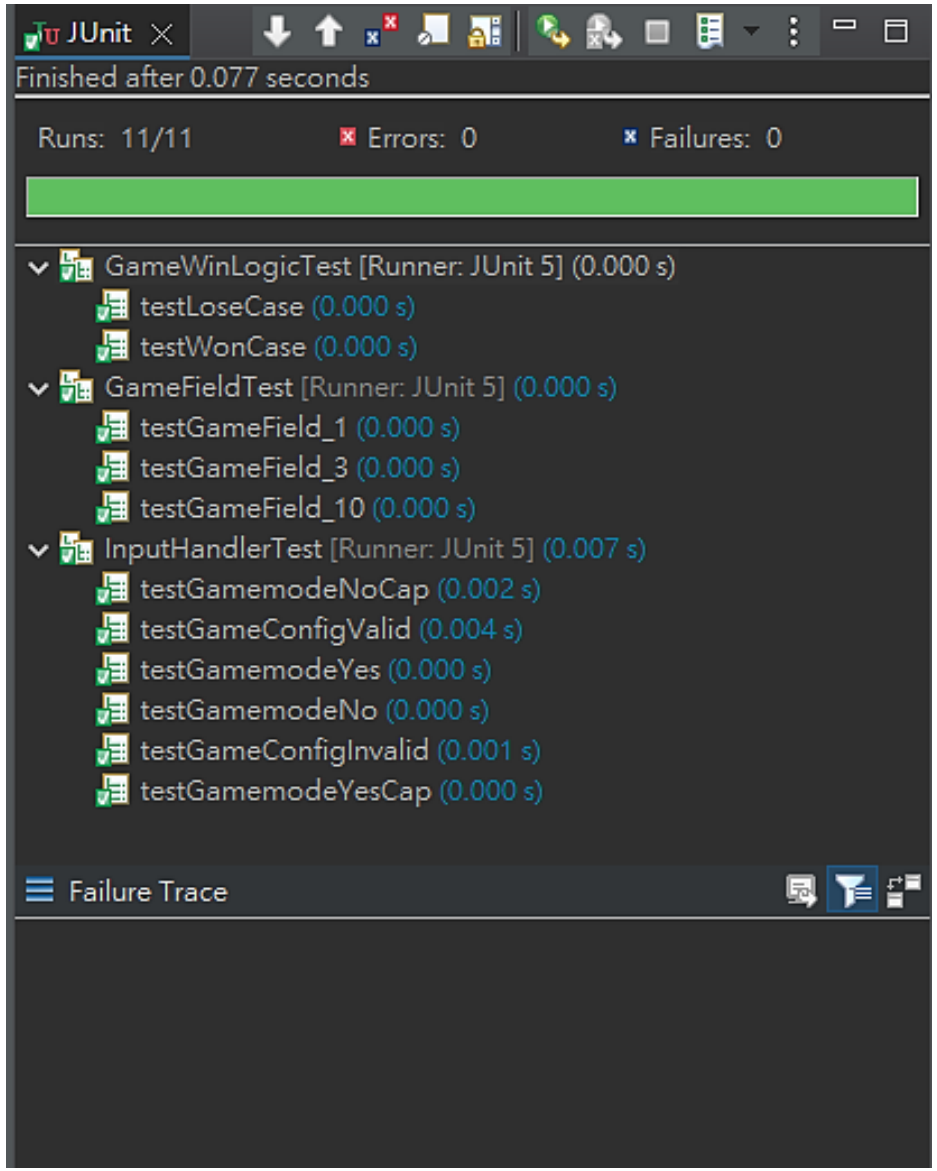
```java
private static int[][] createMineField(int rows, int cols, int mines) {
    int[][] field = new int[rows][cols];
    if (mines >= rows * cols) {
        mines = rows * cols;
    }
    int minesPlaced = 0;
    while (minesPlaced < mines) {
        Point random = getRandomCell(field);
        while (field[random.x][random.y] == MINE) {
            random = getRandomCell(field);
        }
        field[random.x][random.y] = MINE;
        minesPlaced++;
    }
    setHint(field);
    return field;
}
```

Smells

```java
private static void setHint(int[][] field) {
    int lastRow = field.length - 1;
    for (int r = 0; r < field.length; r++) {
        int lastCol = field[r].length - 1;
        for (int c = 0; c < field[r].length; c++) {
            // Don't set a hint if the cell is a mine.
            if (field[r][c] == -1) {
                continue;
            }

            // Otherwise, count adjacent mines.
            int count = 0;
            if (r != 0 && field[r - 1][c] == -1) {
                count++;
            }
            if (r != 0 && c != lastCol && field[r - 1][c + 1] == -1) {
                count++;
            }
            if (c != lastCol && field[r][c + 1] == -1) {
                count++;
            }
            if (r != lastRow && c != lastCol && field[r + 1][c + 1] == -1) {
                count++;
            }
            if (r != lastRow && field[r + 1][c] == -1) {
                count++;
            }
            if (r != lastRow && c != 0 && field[r + 1][c - 1] == -1) {
                count++;
            }
            if (c != 0 && field[r][c - 1] == -1) {
                count++;
            }
            if (r != 0 && c != 0 && field[r - 1][c - 1] == -1) {
                count++;
            }

            // Set the value of the cell to the number of adjacent mines.
            field[r][c] = count;
        }
    }
}
```

```java
private static void setHint(int[][] field) {
    int[] directions = {-1, 0, 1}; // 相對位置
    for (int r = 0; r < field.length; r++) {
        for (int c = 0; c < field[r].length; c++) {
            if (field[r][c] == MINE) continue;

            int count = 0;
            for (int dr : directions) {
                for (int dc : directions) {
                    if (dr == 0 && dc == 0) continue;
                    int nr = r + dr, nc = c + dc;
                    if (nr >= 0 && nr < field.length && nc >= 0 && nc < field[r].length && field[nr][nc] == -1) {
                        count++;
                    }
                }
            }
            field[r][c] = count;
        }
    }
}
```

單元測試結果


JUnit
Finished after 0.077 seconds
Runs: 11/11
Errors: 0
Failures: 0
GameWinLogicTest [Runner: JUnit 5] (0.000 s)
testLoseCase (0.000 s)
testWonCase (0.000 s)
GameFieldTest [Runner: JUnit 5] (0.000 s)
testGameField_1 (0.000 s)
testGameField_3 (0.000 s)
testGameField_10 (0.000 s)
InputHandlerTest [Runner: JUnit 5] (0.007 s)
testGamemodeNoCap (0.002 s)
testGameConfigValid (0.004 s)
testGamemodeYes (0.000 s)
testGamemodeNo (0.000 s)
testGameConfigInvalid (0.001 s)
testGamemodeYesCap (0.000 s)
Failure Trace

心得

- 實際操作是真正學會本門課的重點
- 為重構沒有標準答案
- 測試的重要性
- 留下乾淨程式的重要性